INSTITUTO SUPERIOR TECNOLÓGICO QUITO

Tercer Módulo – Python, Estructuras de Control

python™

Carlos Alberto Pucha

1 Estructuras de control

1.1 Selección (sentencias condicionales)

- Selección de una de varias alternativas en base a alguna condición.
- Indentación para estructurar el código
- Importante el símbolo ':'

[7]:
```
 # # Asigno el bvalo uno a la variable x
x = int(input("Ingrese el valor de x"))

# Implemento una condicional para saber si x es mayor a 0
if x > 0:
    print('Valor positivo')
else:
    print('Valor no positivo')
```

```
---------------------------------------------------------------------------
KeyboardInterrupt                         Traceback (most recent call last)
Cell In[7], line 2
      1  # # Asigno el bvalo uno a la variable x
----> 2 x = int(input("Ingrese el valor de x"))
      4 # Implemento una condicional para saber si x es mayor a 0
      5 if x > 0:

File D:\anaconda\envs\jupyterlab-introduccionpython\Lib\site-packages\ipykernel\kernelbase.py:1275, in Kernel.raw_input(self, prompt)
   1273     msg = "raw_input was called, but this frontend does not support input requests."
```

```
   1274     raise StdinNotImplementedError(msg)
-> 1275 return self._input_request(
   1276     str(prompt),
   1277     self._parent_ident["shell"],
   1278     self.get_parent("shell"),
   1279     password=False,
   1280 )

File D:\anaconda\envs\jupyterlab-introduccionpython\Lib\site-pack
ages\ipykernel\kernelbase.py:1320, in Kernel._input_request(self,
prompt, ident, parent, password)
   1317 except KeyboardInterrupt:
   1318     # re-raise KeyboardInterrupt, to truncate traceback
   1319     msg = "Interrupted by user"
-> 1320     raise KeyboardInterrupt(msg) from None
   1321 except Exception:
   1322     self.log.warning("Invalid Message:", exc_info=True)

KeyboardInterrupt: Interrupted by user
Ingrese el valor de x 5
```

- Las condiciones son expresiones que se evalúan a True o False.
- Operadores de comparación, lógicos, de identidad y de pertenencia vistos anteriormente.

[ ]:
```
x = 3
y = [1, 2, 3]

print(x > 0)

print(x > 0 and x < 10)
print(x is not y)
print(x in y)
```

- Se pueden introducir más 'ramas' en sentencias condicionales a través de la palabra clave elif.

[ ]:
```
x = -3

if x > 0:

    print('Valor positivo')
elif x == 0:
    print('Valor nulo')
else:
    print('Valor negativo')
```

- Anidamiento.

[ ]:
```
val = 120
if val > 0:
    if val < 100:
        print('Valor positivo')
    else:
        print('Valor muy positivo')
```

```
        print('Valor muy positivo')
else:
    print('Valor negativo')
```

EJERCICIOS sentencias condicionales

1. Escribir un programa en Python que pregunte al usuario la edad por teclado y muestre en pantalla si es mayor de edad o no. Es mayor de edad cuando la edad es mayor o igual a 18

[ ]:
```
# Ingrese la edad

edad = int(input("Ingrese su edad: "))

if edad >= 18:
    print("Es mayor de edad")
else:
    print("No es mayor de edad")
```

2. Escribir un programa en Python donde el usuario ingrese 2 números por teclado y muestre por pantalla la division de los dos números. Si el divisor es cero, el programa debe mostrar el mensaje "No se puede dividir un valor para cero".

[ ]:
```
# Ingreso de los dos números por teclado

num1= float(input("Ingrese el primer número: "))
num2= float(input("Ingrese el primer número: "))

# Verificación del divisor y realización de la división

if num2 != 0:
    print("El resultado de la división es:", num1 / num2)
else:
    print("No se puede dividir un valor para cero")
```

3. Escribir un programa en Python que pida al usuario un numero por teclado y muestre en pantalla si el numero es par o impar.

[ ]:
```
# Ingreso del número por teclado

numero = int(input("Ingrese un número: "))

# Verificación si el número es par o impar

if numero % 2 == 0:
    print("El número es par")
else:
    print("El número es impar")
```

2 Switch

[ ]:
```
a = -6

match a:
    case _ if a > 0:
        print('positive')
    case _ if a == 0:
        print('zero')
    case _ if a < 0:
        print('negative')
```

[ ]:
```
a = 'l' #week day

match a:
    case 'L':
        print("Lunes")
    case 'M':
        print("Martes")
    case 'X':
        print("Miercoles")
    case _:
        print("Wrong Day")
```

Ejercicio

1. Crear un programa en Python donde el usuario ingrese por teclado, el tipo de operaación aritmetica a realizar, ejemplo suma, resta, multiplicacion y division, ademas el usuario debe ingresar dos numeros enteros y realizarr la operacion que ingreso el usuario. Para el programa solo debe utilizar el condicional match

[ ]:
```
# ingreso de datos
operacion = input("Ingrese la operación que desea realizar puede se
num1= int(input("Ingrese el primer número entero: "))
num2= int(input("Ingrese el segundo número entero: "))

# Uso del condicional match
match operacion:
    case "suma":
        print("Resultado:", num1 + num2)
    case "resta":
        print("Resultado:", num1 - num2)
    case "multiplicación":
        print("Resultado:", num1 * num2)
    case "division":
        if num2 != 0:
            print("Resultado", num1 / num2)
        else:
            print("Error: no se puede dividir para cero")
    case _:
        print("Operación no válida")
```

https://github.com/AnbuDigital-Ec/introduccionpythonnocturno/blob/main/sentenciasdecontrol.ipynb

Expresión ternaria

- Estructura if-else condensado en una línea.
- Se recomienda usar solo en casos sencillos.

[ ]:
```
# # # Ejemplo: cálculo del valor absoluto de un número.

x = -3
resultado = x if x >= 0 else-x
print(resultado)
```

[ ]:
```
val =-3

if val >=0:
    resultado = val
else:
    resultado =-val

print(resultado)
```

Concatenación de comparaciones

- Se pueden concatenar comparaciones en una misma expresión:
  – and: true, si ambos son true.
  – or: true, si al menos uno es true.
  – not: inversión del valor de verdad de una expresión.
- 'elif' para comparar tras un 'if' Enlace a Tablas de Verdad.

[ ]:
```
genero = 'Drama' fecha_de_estreno = 1989
if genero == 'Comedia' or genero == 'Acción':
    print('Buena película!')
elif fecha_de_estreno >= 1990 and fecha_de_estreno < 2000:
    print('Buena década!')
else:
print('Meh')
```

Comparación de variables: '==' vs 'is'

- '==' compara si el valor de las variables es el mismo
- 'is' compara si los objetos en las variables son iguales (referencia)

[ ]:
```
a = 1000
b = a

print(a is b)
print(a == b)

c=type(a)
```

```
print(c)
d=type(b)
print(d)

print(c==d)

print(id(a))
print(id(b))
```

[ ]:
```
a = [1, 2, 3]
b = [1, 2, 3]
c = a

print(a is b)
print(a is c)
print(a == b)

print(id(a))
print(id(b))
print(id(c))
```

Valor booleano

- Todos los objetos en Python tiene inherentemente un valor booleano: True o False.
- Cualquier número distinto de 0 ó cualquier objeto no vacío tienen valor True.
- El número 0, objetos vacíos y el objeto especial None tienen valor False.

[ ]:
```
a = -10 #True
b = None    #False

if a:
    print("a es True")
if not b:
    print("b es False")
```

[ ]:
```
c = [1,2,3]
# a c le reasigno None
c = None #0 es igual a vacio a nada
if c:
print("Lista no vacía")
    else:
print("Lista vacía")
```

Iteración (bucles)

- Repetición de un bloque de código.
- La terminación del bucle depende del tipo de bucle.

- Dos tipos: while y for.

Bucle 'while'

- Repetición de un bloque de código hasta que se deje cumplir una expresión (es decir, hasta que una condición evalue a False).
- Si la condición evalua a False desde el principio, el bloque de código nunca se ejecuta.
- Cuidado con los bucles infinitos.

Formato general:

while test: # Mientras se cumple la condición statements # Instrucciones a ejecutar

[ ]:
```
# # Ejemplo: Mostrar los primeros 3 objetos de una lista
indice = 0
numeros = [9, 4, 7, 1, 2]
while indice < 3: print(numeros[indice]) indice =indice+1
```

[ ]:
```
# Ejemplo: contar y mostrar los números inferiores a 10.
numeros = [33, 3, 9, 21, 1, 7, 12, 10, 8]
contador = 0
indice = 0
while indice < len(numeros):
if numeros[indice] < 10: print(numeros[indice]) contador += 1
indice += 1
print('Contador:', contador) print('Indice:', indice)
```

[ ]:
```
nombre = 'Pablo'
while nombre: # Mientras 'nombre' no sea vacío
    print(nombre)
    nombre = nombre[1:]
```

[ ]:
```
# #bucle infinito
i = 0
while i < 10:
    print(i) i += 1
```

Ejercicio 1: Desarrollar un programa en python donde se pida al usuario un número entero positivo y muestre por pantalla todos los números impares desde el 1 hasta ese número separado por comas

[ ]:
```
# Programa que muestra los números impares desde 1 hasta un número

# Pido al usuario un número entero
n = int(input("Ingrese un número entero positivo: "))

# Verifico que el número sea positivo
if n <= 0:
    print("Error: debe ingresar un número entero positivo mayor que
else:
```

```
    impares = []  # Lista para almacenar los números impares

    # Recorro desde 1 hasta n
    for i in range(1, n + 1):
        if i % 2 != 0:          # Verifico si el número es impar
            impares.append(str(i))  # Convierto a texto y lo agrego

    # Muestro los números separados por comas
    print("Números impares desde 1 hasta", n, ":")
    print(", ".join(impares))
```

Ejercicio 2: Crea un programa en python donde el usuario ingrese un número el programa pregunte al usuario Si desea continuar sí o no Si el usuario ingresa así el programa debe volver a pedir otro número Si el usuario dice no el programa termina. Mostrar la suma de todos los números ingresados.

[10]:
```
# Programa que suma números hasta que el usuario decida no continua

suma = 0  # Acumulador
continuar = "si"

while continuar.lower() == "si":
    # Pido un número al usuario
    numero = float(input("Ingrese un número: "))
    suma += numero  # Sumo el número ingresado

    # Pregunto si desea continuar
    continuar = input("¿Desea continuar? (si/no): ").strip().lower(

    # Valido la respuesta
    while continuar != "si" and continuar != "no":
        print("Respuesta no válida. Escriba 'si' o 'no'.")
        continuar = input("¿Desea continuar? (si/no): ").strip().lo

# Muestro el resultado final
print("La suma de todos los números ingresados es:", suma)
```

```
Ingrese un número:  4
¿Desea continuar? (si/no):  4
Respuesta no válida. Escriba 'si' o 'no'.
¿Desea continuar? (si/no):  si
Ingrese un número:  5
¿Desea continuar? (si/no):  si
Ingrese un número:  555
¿Desea continuar? (si/no):  no
La suma de todos los números ingresados es: 564.0
```

https://github.com/AnbuDigital-Ec/introduccionpythonnocturno/blob/main/sentenciasdecontrol.ipynb

Bucle 'for'

- Permite recorrer los items de una sequencia o un objeto iterable.
- Funciona en strings, listas, tuplas, etc.

Formato general:
for item in objeto: # Asigna los items del objeto a la variable item en cada iteración
statements # Instrucciones a ejecutar

[ ]:
```
peliculas = ['Matrix', 'The purge', 'Avatar', 'Star Wars']
for pelicula in peliculas:
    print(pelicula)
```

- También se usa para iterar un número preestablecido de veces (counted loops):

[ ]:
```
for i in range(10):
    print(i)
```

- range es útil en combinación con len porque permite acceder a los elementos de una secuencia por posición.

[ ]:
```
nombre = 'Pablo'

for i in range(len(nombre)):
    print(nombre[i])
```

[ ]:
```
for letra in nombre:
    print(letra)
```

- Iteración de tuplas:

[ ]:
```
tuplas = [(1, 2,4), (3, 4), (5, 6)]
for a, b,c in tuplas:
    print(a, b,c)
```

- Iteración de diccionarios. Se iteran las claves:

[ ]:
```
diccionario = {'a': 1, 'b': 2, 'c': 3}

for key in diccionario:
    print(f"{key} => {diccionario[key]}")
```

- Para iterar los pares (clave-valor) o únicamente los valores, se deben usar los métodos items y values.

[ ]:
```
for key, value in diccionario.items():
    print(f"{key} => {value}")
```

[ ]:
```
for value in diccionario.values():
    print(value)
```

- La variable item en la cabecera del for puede ser cualquier expresión que sea válida como parte izquierda de una asignación convencional.

[ ]:
```
for a, b, c in [(1, 2, 3), (4, 5, 6)]:
    print(a, b, c)
```

Las sentencias break, continue y else

Break y continue

- Solo tienen sentido dentro de bucles.
- Break permite terminar el bucle por completo.
- Continue permite saltar a la siguiente iteración, continuando con el bucle.
- Pueden aparecer en cualquier parte de un bucle, pero normalmente aparecen dentro de sentencias condicionales (if).

[ ]:
```
# # Ejemplo: break
rating_to_find = 4.2
movie_ratings = [4.3, 2.5, 1.7, 4.2, 3.8, 3.3, 4.5]

for rating in movie_ratings:
    print(rating)
    if rating == rating_to_find:
        print("Found")
        break
print(rating)
```

[ ]:
```
for i in range(10):
    if i % 2 == 0:
        continue # Si el número es par, salta a la siguiente iterac
    print(f'Odd number {i}')
```

- Observa como la sentencia continue te puede ayudar a reducir el número de niveles de anidamiento.
- Sin continue el anterior ejemplo sería:

[ ]:
```
for i in range(10):
    if i % 2 != 0:
        print('Numero impar:', end=' ')
        print(i)
```

[ ]:
```
for i in range(5):
    for a in range(2):
        print(f"{i} - {a}")
        if i == 3 and a == 0:
            break
```

Else

- Los bucles pueden tener una sensencia else.
- Resulta poco intuitiva para muchos programadores porque esta

sintaxis no existe en otros lenguajes.

- Se ejecuta cuando el bucle termina con normalidad; es decir, cuando no termina a causa de un break.

[ ]:
```
lista = [60,60,30,60]
element_to_find = 60

for element in lista:
    if element == element_to_find:
        print("found")
        break
else:
    print("not found")
```

[ ]:
```
# with flag
lista = [10,30,50,40]
element_to_find = 30
found = False

for element in lista:
    if element == element_to_find:
        found = True
        break
if found:
    print("dato encontrado")
else:
    print("not found")
```

Ejercicio 1: Crear un programa en python donde el usuario ingrese por teclado la siguiente información: Cantidad en dólares a invertir (tipo flotante) Interés anual El número de años que se va a invertir. El programa debe mostrar por pantalla el capital obtenido en la inversión cada año que dura la inversión.

[9]:
```
# Programa que calcula el capital obtenido en una inversión cada añ

# Entrada de datos
capital_inicial = float(input("Ingrese la cantidad en dólares a inv
interes_anual = float(input("Ingrese el interés anual (%): "))
anios = int(input("Ingrese el número de años de la inversión: "))

# Validación básica
if capital_inicial <= 0 or interes_anual < 0 or anios <= 0:
    print("Error: los valores ingresados deben ser positivos.")
else:
    capital = capital_inicial

    print("\nCapital obtenido por año:")

    # Cálculo del capital para cada año
    for año in range(1, anios + 1):
        capital = capital * (1 + interes_anual / 100)
        print(f"Año {año}: ${capital:.2f}")
```

```
Ingrese la cantidad en dólares a invertir:  58744
Ingrese el interés anual (%):  8
```

```
Ingrese el número de años de la inversión:  7

Capital obtenido por año:
Año 1: $63443.52
Año 2: $68519.00
Año 3: $74000.52
Año 4: $79920.56
Año 5: $86314.21
Año 6: $93219.35
Año 7: $100676.89
```

Ejercicio 1: Desarrollar un programa en python donde el usuario ingrese un número entero positivo, verificar si el número ingresado es primo o no. Un número primo es cuando divisible para 1 y para si mismo

[8]:
```python
# Programa que verifica si un número es primo

# Entrada de datos
numero = int(input("Ingrese un número entero positivo: "))

# Validación
if numero <= 0:
    print("Error: debe ingresar un número entero positivo.")
elif numero == 1:
    print("El número 1 NO es primo.")
else:
    es_primo = True  # Variable bandera

    # Verifico divisores desde 2 hasta numero - 1
    for i in range(2, numero):
        if numero % i == 0:
            es_primo = False
            break  # Salgo del ciclo si encuentro un divisor

    # Resultado
    if es_primo:
        print(f"El número {numero} ES primo.")
    else:
        print(f"El número {numero} NO es primo.")
```

```
Ingrese un número entero positivo:  7
El número 7 ES primo.
```

https://github.com/AnbuDigital-Ec/introduccionpythonnocturno/blob/main/sentenciasdecontrol.ipynb

List Comprehensions

- Permiten construir listas a través de la ejecución repetida (sentencia for) de una expressión para cada item de un objeto iterable.
- Van entre '[' y ']'. Esto es indicativo de que estamos construyendo una lista.

Sintaxis:

[for in] Ejemplo: repetir los carácteres de un string.

[ ]:
```
[char*3 for char in "Enrique"]
```

[1]:
```
lista = []
for char in 'Enrique':
    lista.append(char)
print(lista)
```

```
['E', 'n', 'r', 'i', 'q', 'u', 'e']
```

Comúnmente, el item de la sentencia for aparecerá en la expresión principal, pero eso no es obligatorio.

[ ]:
```
[2 for _ in 'Enrique']
```

4.0.1 Versión extendida

Se puede especificar un filtro (sentencia if) para obtener únicamente los elementos que cumplan cierta condición.

Sintaxis:
[for in if] Ejemplo: obtener números pares:

[ ]:
```
[y for y in range(9) if y % 2 == 0]
```

- Las list comprehension no son realmente requeridas, ya que siempre podemos escribir un bucle equivalente.

[ ]:
```
pares = []
for x in range(9):
    if x % 2 == 0:
        pares.append(x)
print(pares)
```

4.0.2 Versión completa

La sentencia if de una list comprehension también puede contener una expresión alternativa.

Sintaxis:
[if else for in] Ejemplo: poner a cero los números pares.

[ ]:
```
[0 if x % 2 == 0 else x for x in range(9)]
```

[ ]:
```
[x**2 if x > 2 else x for x in range(-4,5) if x > 0]
```

[ ]:
```
lista = []
for x in range(-4,5):
    if x > 0:
        if x > 2:
            lista.append(x**2)
        else:
            lista.append(x)
print(lista)
```

[ ]:
```
[x**2 if x > 0 else 100 if x==0 else x for x in range(-4,5)]
```

[ ]:
```
lista = []
for x in range(-4,5):
    if x > 0:
        lista.append(x**2)
    elif x == 0:
        lista.append(100)
    else:
        lista.append(x)
print(lista)
```

4.0.3 Anidamiento

Las list comprehensions soportan anidamiento en sus expresiones. Ejemplo: bucle anidado para obtener una lista de tuplas que combinan los elementos de dos listas dadas.

[ ]:
```
lista_1 = [1, 2, 3]
lista_2 = [4, 5]
[(x, y) for x in lista_1 for y in lista_2]
```

4.0.4 Pros y contras

- Principales ventajas de las comprehensions en comparación a un bucle convencional:

– Expresión compacta y legible, si estás familiarizado con la sintaxis.
– Mejor rendimiento.

- Desventaja: no escalan bien. Una list comprehension se puede convertir rápidamente en una expresión difícil de entender.

4.0.5 Otros tipos de comprehensions

Dictionary comprehensions

Usando '{' y '}' como delimitadores y una expresión 'clave : valor', se obtiene un diccionario en lugar de una lista. Ejemplo: diccionario donde cada valor es el cuadrado de la clave.

[ ]:
```
d = {x : x*x for x in range(10)}
print(d)
print(type(d))
```

[ ]:
```
items = ['Banana', 'Pear', 'Olives']
price = [1.1, 1.4, 2.4]
shopping = {k:v for k,v in zip(items,price)}
print(shopping)
```

[ ]:
```
items = ['Banana', 'Pear', 'Olives']
price = [1.1, 1.4, 2.4]
for k in zip(items, price):
```

```
    print(type(k))
    print(k)
```

Set comprehensions

Usando '{' y '}' como delimitadores y una expresión simple (al igual que en las list comprehensions), se obtiene un conjunto. Ejemplo: conjunto que incluye los 10 primeros números naturales.

[ ]:
```
# c = {x for x in range(10)}
# print(c)
# print(type(c))
```

Tuple comprehensions

[ ]:
```
# tupla = tuple(x for x in range(4))
# print(tupla)
```

5 Excepciones

- Las excepciones son eventos que representan situaciones excepcionales.
- Alteran el flujo de ejecución convencional.
- Python lanza excepciones automáticamente cuando se producen errores.
- El programador puede lanzar excepciones de manera explícita y también capturar excepciones para actuar como se crea conveniente.

Try/except

- Permite capturar excepciones y actuar en consecuencia. Ejemplo: error de acceso fuera de rango.

[ ]:
```
# lista = [6, 1, 0, 5]
# lista[4]
# print('Código tras el error')
```

[ ]:
```
# lista = [6, 1, 0, 5]
# i = 300
# try:
#    a = lista[i]
# except IndexError:
#    print('He capturado la excepción de tipo IndexError')
#    a = 0
# print('Código tras el bloque try')
# print(a)
```

- Normalmente, al capturar excepciones, querremos ser lo más específicos posible, pero también se puedes usar una sentencia try-except que capture cualquier error.

[ ]:
```
# try:
```

```
#     4/0
# except:
#     print('He capturado la división por cero')
```

- Recoger la excepción e imprimir el mensaje de error

[ ]:
```
# lista = [6, 1, 0, 5]
# try:
#     print(lista[4])
# except Exception as e:
#     print("Error = " + str(e))
# print('Reacheable Code')
```

Try/finally

- A través de finally podemos especificar código que queremos que se ejecute siempre (independientemente de si se produce la excepción o no).
- Se suele usar para liberar recursos.

[ ]:
```
# lista = [6, 1, 0, 5]
# try:
#     a = lista[1]
# except IndexError:
#     print('Exception IndexError Captured')
# finally:
#     print('Bloque finally')
#     b = lista[2]
# print('Código tras el bloque try')
# print(b)
```

raise

- La sentencia raise nos permite lanzar excepciones de manera explícita.

[2]:
```
# lista = [6, 1, 0, 5]
# indice = 4
# try:
#     if indice >= len(lista):
#         raise IndexError('Índice fuera de rango')
# except IndexError:
#     print('Excepción de tipo IndexError capturada')
```

5.1 Ejercicios

1. Escribe un programa que calcule la suma de todos los elementos de una lista dada. La lista sólo puede contener elementos numéricos.

[3]:
```
# Función que calcula la suma de los elementos de una lista
def suma_lista(lista):
    suma = 0  # Acumulador
```

```
    for elemento in lista:
        suma += elemento

    return suma


# Lista de ejemplo (solo números)
numeros = [5, 10, 3, 7, 2]

# Llamada a la función
resultado = suma_lista(numeros)

# Mostrar el resultado
print("La suma de los elementos de la lista es:", resultado)
```

```
La suma de los elementos de la lista es: 27
```

2. Dada una lista con elementos duplicados, escribir un programa que muestre una nueva lista con el mismo contenido que la primera pero sin elementos duplicados. Para este ejercicio, no puedes hacer uso de objetos de tipo 'Set'.

[1]:
```
# Función que elimina los elementos duplicados de una lista
def eliminar_duplicados(lista):
    lista_sin_duplicados = []

    for elemento in lista:
        if elemento not in lista_sin_duplicados:
            lista_sin_duplicados.append(elemento)

    return lista_sin_duplicados


# Lista de ejemplo con elementos duplicados
lista_original = [1, 2, 3, 2, 4, 1, 5, 3]

# Llamada a la función
resultado = eliminar_duplicados(lista_original)

# Mostrar resultados
print("Lista original:", lista_original)
print("Lista sin duplicados:", resultado)
```

```
Lista original: [1, 2, 3, 2, 4, 1, 5, 3]
Lista sin duplicados: [1, 2, 3, 4, 5]
```

3. Escribe un programa que construya un diccionario que contenga un número (entre 1 y n) de elementos de esta forma: (x, x*x). Ejemplo: para n = 5, el diccionario resultante sería {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}

[2]:
```
# Función que construye un diccionario con pares (x, x*x)
def construir_diccionario(n):
    diccionario = {}
```

```
    for x in range(1, n + 1):
        diccionario[x] = x * x

    return diccionario


# Entrada de datos
n = int(input("Ingrese un número entero positivo: "))

# Validación
if n <= 0:
    print("Error: el número debe ser mayor que cero.")
else:
    # Llamada a la función
    resultado = construir_diccionario(n)

    # Mostrar resultado
    print("Diccionario resultante:")
    print(resultado)
```

```
Ingrese un número entero positivo:  5
Diccionario resultante:
{1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```

4. Escribe un programa que, dada una lista de palabras, compruebe si alguna empieza por 'a' y tiene más de 9 caracteres. Si dicha palabra existe, el programa deberá terminar en el momento exacto de encontrarla. El programa también debe mostrar un mensaje apropiado por pantalla que indique el éxito o el fracaso de la búsqueda. En caso de éxito, también se mostrará por pantalla la palabra encontrada.

[4]:
```
# Función que busca una palabra que empiece por 'a' y tenga más de
def buscar_palabra(lista_palabras):
    for palabra in lista_palabras:
        if palabra.lower().startswith('a') and len(palabra) > 9:
            return palabra  # Termina en el momento exacto de encon
    return None


# Lista de palabras de ejemplo
palabras = [
    "casa",
    "avion",
    "automovil",
    "astronauta",
    "biblioteca",
    "arbol"
]

# Llamada a la función
resultado = buscar_palabra(palabras)

# Mostrar resultado
```

```
if resultado:
    print("Búsqueda exitosa.")
    print("Palabra encontrada:", resultado)
else:
    print("Búsqueda fallida. No se encontró ninguna palabra que cum
```

```
Búsqueda exitosa.
Palabra encontrada: astronauta
```

5. Dada una lista L de números positivos, escribir un programa que muestre otra lista (ordenada) que contenga todo índice i que cumpla la siguiente condición: L[i] es múltiplo de 3. Por ejemplo, dada la lista L = [3,5,13,12,1,9] el programa mostrará la lista [0,3,5] dado que L[0], L[3] y L[5] son, respectivamente, 3, 12 y 9, que son los únicos múltiplos de 3 que hay en L.

[5]:
```
# Lista de números positivos
L = [3, 5, 13, 12, 1, 9]

# Lista para almacenar los índices
indices = []

# Recorrer la lista usando índices
for i in range(len(L)):
    if L[i] % 3 == 0:
        indices.append(i)

# Mostrar el resultado
print("Índices cuyos valores son múltiplos de 3:", indices)
```

```
Índices cuyos valores son múltiplos de 3: [0, 3, 5]
```

6. Dado un diccionario cuyos elementos son pares de tipo string y numérico (es decir, las claves son de tipo 'str' y los valores son de tipo 'int' o 'float'), escribe un programa que muestre por pantalla la clave cuyo valor asociado representa el valor númerico más alto de todo el diccionario. Por ejemplo, para el diccionario {'a': 4.3, 'b': 1, 'c': 7.8, 'd': -5} la respuesta sería 'c', dado que 7.8 es el valor más alto de los números 4.3, 1, 7.8 y -5.

[6]:
```
# Diccionario dado
diccionario = {'a': 4.3, 'b': 1, 'c': 7.8, 'd': -5}

# Inicializamos variables
mayor_valor = None
clave_mayor = None

# Recorremos el diccionario
for clave, valor in diccionario.items():
    if mayor_valor is None or valor > mayor_valor:
        mayor_valor = valor
```

```
        clave_mayor = clave

# Mostramos el resultado
print("La clave con el valor numérico más alto es:", clave_mayor)
```

```
La clave con el valor numérico más alto es: c
```

7. Dada la lista a = [2, 4, 6, 8] y la lista b = [7, 11, 15, 22], escribe un programa que itere las listas a y b y multiplique cada elemento de a que sea mayor que 5 por cada elemento de b que sea menor que 14. El programa debe mostrar los resultados por pantalla.

[7]:
```
# Listas dadas
a = [2, 4, 6, 8]
b = [7, 11, 15, 22]

# Recorremos la lista a
for elemento_a in a:
    if elemento_a > 5:

        # Recorremos la lista b
        for elemento_b in b:
            if elemento_b < 14:
                resultado = elemento_a * elemento_b
                print(f"{elemento_a} x {elemento_b} = {resultado}")
```

```
6 x 7 = 42
6 x 11 = 66
8 x 7 = 56
8 x 11 = 88
```

8. Escribir un programa que pida un valor numérico X al usuario. Para ello podéis hacer uso de la función predefinida 'input'. El programa deberá mostrar por pantalla el resultado de la división 10/X. En caso de que el usuario introduzca valores no apropiados, el programa deberá gestionar correctamente las excepciones, por ejemplo, mostrando mensajes informativos por pantalla.

[8]:
```
try:
    # Solicitar el valor al usuario
    x = float(input("Ingrese un valor numérico X: "))

    # Realizar la división
    resultado = 10 / x

    # Mostrar el resultado
    print("El resultado de 10 / X es:", resultado)

except ValueError:
    print("Error: debe ingresar un valor numérico válido.")

except ZeroDivisionError:
    print("Error: no se puede dividir para cero.")
```

```
    print("Error: no se puede dividir para cero.")

except Exception as e:
    print("Ha ocurrido un error inesperado:", e)
```

```
Ingrese un valor numérico X:  55
El resultado de 10 / X es: 0.18181818181818182
```

9. Escribir un programa que cree un diccionario cualquiera. Posteriormente, el programa pedirá al usuario (a través de la función predefinida 'input') que introduzca una clave del diccionario. Si la clave introducida es correcta (es decir, existe en el diccionario), el programa mostrará por pantalla el valor asociado a dicha clave. En caso de que la clave no exista, el programa gestionará de manera apropiada el error, por ejemplo, mostrando un mensaje informativo al usuario.

[9]:
```
# Diccionario creado
diccionario = {
    "nombre": "Carlos",
    "edad": 30,
    "ciudad": "Quito",
    "profesion": "Ingeniero"
}

# Solicitar la clave al usuario
clave = input("Ingrese una clave del diccionario: ")

# Verificar si la clave existe
if clave in diccionario:
    print("El valor asociado a la clave es:", diccionario[clave])
else:
    print("Error: la clave ingresada no existe en el diccionario.")
```

```
Ingrese una clave del diccionario:  n
Error: la clave ingresada no existe en el diccionario.
```

10. Escribe una list comprehension que construya una lista con los números enteros positivos de una lista de números dada. La lista original puede incluir números de tipo float, los cuales deben ser descartados.

[10]:
```
# Lista original
lista = [3, -2, 4.5, 7, 0, -1, 6.0, 9, 2.3]

# List comprehension para enteros positivos
lista_enteros_positivos = [x for x in lista if isinstance(x, int) a

# Mostrar resultado
print("Lista original:", lista)
print("Lista de enteros positivos:", lista_enteros_positivos)
```

```
Lista original: [3, -2, 4.5, 7, 0, -1, 6.0, 9, 2.3]
Lista de enteros positivos: [3, 7, 9]
```

11. Escribe una set comprehension que, dada una palabra, construya un conjunto que contenga las vocales de dicha palabra.

[11]:
```
# Solicitar una palabra al usuario
palabra = input("Ingrese una palabra: ")

# Set comprehension para obtener las vocales
vocales = {letra for letra in palabra.lower() if letra in "aeiou"}

# Mostrar resultado
print("Vocales encontradas:", vocales)
```

```
Ingrese una palabra:  fgsd
Vocales encontradas: set()
```

12. Escribe una list comprehension que construya una lista con todos los números del 0 al 50 que contengan el dígito 3. El resultado será: [3, 13, 23, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 43].

[12]:
```
# List comprehension para números que contienen el dígito 3
numeros_con_3 = [x for x in range(51) if '3' in str(x)]

# Mostrar resultado
print(numeros_con_3)
```

```
[3, 13, 23, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 43]
```

13. Escribe una dictionary comprehension que construya un diccionario que incluya los tamaños de cada palabra en una frase dada. Ejemplo: el resultado para la frase "Soy un ser humano" será {'Soy': 3, 'un': 2, 'ser': 3, 'humano': 6}

[13]:
```
# Solicitar la frase al usuario
frase = input("Ingrese una frase: ")

# Dictionary comprehension para obtener el tamaño de cada palabra
diccionario_tamanios = {palabra: len(palabra) for palabra in frase.

# Mostrar resultado
print(diccionario_tamanios)
```

```
Ingrese una frase:  Hola mundo
{'Hola': 4, 'mundo': 5}
```

14. Escribe una list comprehension que construya una lista que incluya todos los números del 1 al 10 en orden. La primera mitad se mostrarán en formato numérico; la segunda mitad en texto. Es decir

mostrarán en formato numérico, la segunda mitad en texto. Es decir,

el resultado será: [1, 2, 3, 4, 5, 'seis', 'siete', 'ocho', 'nueve', 'diez'].

[14]:
```
# Diccionario de números en texto
numeros_texto = {
    6: "seis",
    7: "siete",
    8: "ocho",
    9: "nueve",
    10: "diez"
}

# List comprehension solicitada
lista_resultado = [x if x <= 5 else numeros_texto[x] for x in range

# Mostrar resultado
print(lista_resultado)
```

```
[1, 2, 3, 4, 5, 'seis', 'siete', 'ocho', 'nueve', 'diez']
```

https://github.com/AnbuDigital-Ec/introduccionpythonnocturno/blob/main/sentenciasdecontrol.ipynb